

```
'between' => 'The :attribute value :input is not
between :min - :max.',
'in' => 'The :attribute must be one of the
following types: :values',
];
```

Laravel offers the functionality of creating complex validation logic quite easily. E.g. you can create a rule that checks a field only if another field is populated say when collecting personal data and the nature of business is to be populated only if the field type of work: business is selected. Or validate a field only when it is present. Another instance is when you want to validate that is always present but may be empty. Conditional rules are added by the `sometimes` rule. You can give a condition to this rule specifying a different field as in the first example or keep it simple for the required field for the second example.

You may want to take a look at the variety of available validation rules offered by Laravel in their documentation.

Error Handling

Laravel automatically configures the error and exception handling which is under the `App\Exceptions\Handler` class. This is where all triggered exceptions are logged and rendered back to the user.

You control the level of error information shared with the user with the `debug` option in the `config/app.php` configuration file. This option follows the value of the `APP_DEBUG` environment variable, which is stored in the `.env` file. Make sure that this value is set to true in your production environment otherwise you risk exposing sensitive configuration values to your application's end users.

The `report` method under the handler class logs exceptions and it can also send these to an external service. This method also allows you to log an exception but continue handling a request by not rendering an error page. There are some errors that you are not bothered about like a 404 error. The `$dontReport` function will not log in the error and you can define as many classes as you want for this function. The render method converts an exception into an customizable HTTP response that is sent back to the user's browser.

A shortcut to generate the HTTP errors like 404, 401, 500 etc., you can use the `abort` method

```
abort(404, 'Sorry!');
```